MLOps & Industrialisation

Jour 5 — Matin

Julien Rolland

Formation M2 Développement Fullstack

Jour 5

Plan du module

1. Du notebook à la production
2. Sérialiser et optimiser
3. Serving avec FastAPI
4. Docker et CI/CD

Le fossé R&D / Production

Le constat

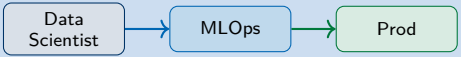
« *It works on my Notebook* » n'est pas une mise en production.

Contraintes du monde réel :

- **Disponibilité :** 1 000 req/s sans s'effondrer
- **Sécurité :** injection de données corrompues, vol de poids du modèle
- **Reproductibilité :** Python 3.10 vs 3.12, PyTorch 2.x vs 1.x

Le rôle du MLOps

Créer un **pont stable** entre le Data Scientist et le DevOps.



Au programme

Sérialisation ▪ Serving ▪ Docker ▪ Monitoring ▪ CI/CD

Sérialisation — sauvegarder un modèle entraîné

Le problème

L'entraînement dure des heures.

Comment **sauvegarder** le modèle sur disque pour le réutiliser en production, sans avoir à tout réentraîner ?

Sérialisation

Convertir l'état interne du modèle (poids, architecture) en une **suite d'octets** écrite dans un fichier.

La désérialisation recrée l'objet en mémoire.

Les approches en Python / PyTorch

- **Pickle** — réflexe natif Python
- **state_dict** — poids seuls (PyTorch)
- **ONNX** — graphe portable universel
- **safetensors** — sans exécution de code

Critères de choix

- Sécurité (source inconnue ?)
- Portabilité (Java, mobile, navigateur ?)
- Performance à l'inférence

Sérialisation (1/2) — le danger de Pickle

Pickle — le réflexe Python

```python
import pickle

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

# Charger -- danger si source inconnue !
with open("model.pkl", "rb") as f:
    model = pickle.load(f)
```

Avantage

Simple, natif, aucune dépendance supplémentaire.

Risque de sécurité critique

Pickle peut **exécuter du code arbitraire** à la désérialisation.

Ne jamais charger un `.pkl` d'origine inconnue.

Rigidité = dette technique

- Nécessite le **même code source** et les mêmes versions de classes
- Incompatible entre versions de Python
- Inutilisable depuis Java, Go, Rust, etc.

Sérialisation (2/2) — ONNX & l'indépendance

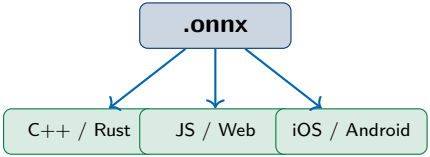
ONNX — Open Neural Network Exchange

Modèle exporté sous forme de **graphe statique figé**.

À l'inférence, **plus besoin de remonter le gradient** : le graphe peut être optimisé par le compilateur (fusion d'opérations, constant folding, quantization).

- **Standard universel :** portable partout
- **Vitesse :** plus rapide que PyTorch brut car aucun overhead d'autograd

```python
import torch.onnx
torch.onnx.export(
    model, dummy_input, "model.onnx",
    input_names=["x"], output_names=["y"],
)
import onnxruntime as ort
sess = ort.InferenceSession("model.onnx")
out = sess.run(None, {"x": x_np})
```



Alternatives PyTorch

- `state_dict` — poids seuls
- `torch.jit.script` — TorchScript
- `safetensors` — sans exécution de code

Model Registry — versionner ses modèles comme son code

Le problème sans registry

- `model_final.onnx`, `model_final_v2.onnx`, `model_ok_cette_fois.onnx`...
- Impossible de savoir quel modèle tourne en prod
- Impossible de rollback proprement

Principe

Un **Model Registry** est à vos modèles ce que Git est à votre code : chaque version est taguée, traçable, et peut être promue ou revertée.

Avec MLflow

```python
import mlflow

with mlflow.start_run():
    mlflow.log_metric("f1", 0.91)
    mlflow.log_param("lr", 1e-4)
    mlflow.onnx.log_model(model, "model")

# Promouvoir en production
client = mlflow.MlflowClient()
client.transition_model_version_stage(
    name="sentiment", version=3,
    stage="Production"
)
```

Metadata à tracer

Dataset utilisé ▪ hyperparamètres ▪ métriques ▪ auteur ▪ date

Optimisation de l'inférence — « Speed is a Feature »

Quantization

Passer les poids de **Float32 à Int8**.

- Gain de vitesse $\times 3$
- Poids du modèle $-75\,\%$
- Perte de précision $< 1\,\%$

Pruning

Supprimer les connexions dont les poids sont proches de zéro.

- Réseau **creux** (*sparse*)
- 50–90 % des poids supprimables
- Fine-tuning nécessaire après

Compilation Hardware

Compiler le graphe pour la carte cible :

- **TensorRT** — GPU NVIDIA
- **OpenVINO** — CPU Intel
- **CoreML** — Apple Silicon

Optimisation en pratique — quand appliquer quoi

Quantization — 3 lignes, impact immédiat

```python
import torch

# Post-Training Dynamic Quantization
model_int8 = torch.quantization.quantize_dynamic(
    model,
    {torch.nn.Linear},    # couches a quantizer
    dtype=torch.qint8
)
# Taille / 4, vitesse x2-3 sur CPU
torch.onnx.export(model_int8, dummy, "model_int8.
    onnx")
```

Quand choisir quoi ?

- **Quantization** — toujours, en premier. Gain immédiat, risque minimal.
- **Pruning** — modèle trop lourd même après quantization. Nécessite un fine-tuning post-pruning.
- **Compilation hardware** — hardware cible fixé et connu à l'avance (GPU NVIDIA en prod, Apple Silicon, etc.). Gain max mais portabilité nulle.

Quantization-Aware Training (QAT)

Si la perte de précision PTQ est trop importante : simuler la quantization **pendant** l'entraînement pour que le modèle s'y adapte.

Serving avec FastAPI — l'architecture Singleton

Anti-pattern à bannir

Charger le modèle **dans** la route API → 500 Mo rechargés à chaque requête → latence + RAM saturée.

Le pattern Singleton

Le modèle est chargé **une seule fois** au démarrage, partagé entre toutes les requêtes. `lifespan` gère le startup et le shutdown proprement.

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = load_model("model.onnx")    # startup : charge une seule fois
    yield
    app.state.model = None                         # shutdown : libere la memoire

app = FastAPI(lifespan=lifespan)

@app.post("/predict")
async def predict(data: InputData):
    return app.state.model.run(data)               # modele deja en memoire, pas de reload
```

Pydantic — typage fort

Valider automatiquement les entrées JSON :

- Évite les crashs sur données malformées
- Erreurs 422 auto-générées avec détail
- Intégration native FastAPI

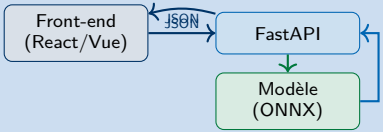
Asynchronisme

`async def` pour ne pas bloquer l'Event Loop pendant les I/O :

- Requêtes BDD, appels HTTP externes
- Inférence : déléguer au `ThreadPoolExecutor`

Swagger UI automatique

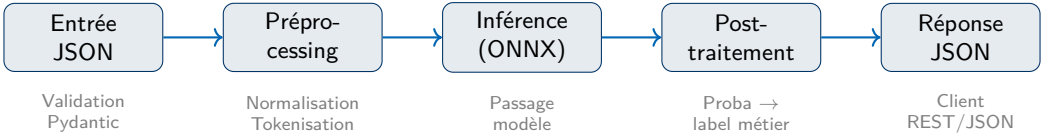
Documentation sur `/docs` — indispensable pour l'intégration Front-end.



Intérêt terrain

Vous connaissez déjà le paradigme REST — FastAPI l'applique à l'IA.

Le pipeline d'inférence complet



Train/Serve Skew — le bug silencieux

Le bug le plus fréquent en prod

Le preprocessing à l'entraînement et celui dans l'API ne font **pas exactement la même chose**.

Le modèle reçoit des données qu'il n'a jamais vues — les prédictions se dégradent **silencieusement**, sans erreur, sans exception.

```
# Entrainement
scaler = StandardScaler()
X_train = scaler.fit_transform(X_raw)

# API -- BUG : nouveau scaler, nouveaux stats
    !
scaler2 = StandardScaler()
X = scaler2.fit_transform([user_input])
```

Solution : un seul objet, versionné avec le modèle

```
# Entrainement : serialiser le scaler
import joblib
scaler = StandardScaler().fit(X_raw)
joblib.dump(scaler, "scaler-v3.pkl")

# API : charger le MEME scaler
scaler = joblib.load("scaler-v3.pkl")
X = scaler.transform([user_input])  # OK
```

Règle d'or

Préprocesseur et modèle sont **un même artefact** : versionnés ensemble, déployés ensemble.

Idéal : empaqueter le preprocessing **dans** le modèle ONNX ou un `sklearn Pipeline`.

L'IA « On-Edge » — zéro serveur, zéro latence

Le problème du serving classique

Chaque requête transite par un serveur :

- Latence réseau incompatible avec la vidéo temps réel
- Coût infrastructure qui explose à l'échelle
- Données sensibles qui quittent l'appareil
- Indisponible hors connexion

On-Edge : l'inférence chez l'utilisateur

- **ONNX Runtime Web** — modèle ONNX dans le navigateur
- **TensorFlow.js** — JavaScript natif
- **MediaPipe / CoreML** — vision sur mobile

Avantages

- **Latence :** pas de round-trip réseau
- **Coût :** le client fournit la puissance de calcul
- **Confidentialité :** données jamais envoyées — RGPD-friendly
- **Résilience :** fonctionne hors connexion

Contraintes

- **RAM et CPU :** mobile $\ll$ serveur — quantization et pruning indispensables
- **Hétérogénéité :** iOS, Android, navigateurs — comportements différents selon le hardware
- **Mise à jour :** impossible de patcher le modèle sans mise à jour de l'app

Containerisation avec Docker

Pourquoi Docker en IA ?

- **Environnement figé :** Python, CUDA, drivers toujours identiques en dev, staging, prod
- **Isolation GPU :** NVIDIA Container Toolkit
- **Reproductibilité :** même image partout

Le problème du poids

Les images Data Science sont lourdes (plusieurs Go) à cause de PyTorch et CUDA.

Solution : images **slim** + ne copier que le nécessaire.

Bonnes pratiques

- Partir de `python:3.10-slim` (pas `:latest`)
- **Multi-stage build** : compiler dans une image lourde, exécuter dans une image légère
- Ne jamais commiter des poids ou des données dans l'image
- Utiliser `.dockerignore`

GPU en prod

`nvidia/cuda:12.1-base` comme base + `-gpus all` au `docker run`.

Dockerfile — optimisation pour l'ingénieur

```
FROM python:3.10-slim AS base
WORKDIR /app

# Layer caching : deps avant le code
COPY requirements.txt .
RUN pip install --no-cache-dir -r
    requirements.txt

# Code source (invalide le cache si change)
COPY src/ ./src/

# Securite : pas de root en prod
RUN useradd --no-create-home appuser
USER appuser

EXPOSE 8000
CMD ["uvicorn", "src.main:app", "--host", "
    0.0.0.0"]
```

Layer caching

Les dépendances changent rarement : les installer **avant** de copier le code.

Docker réutilise le cache si `requirements.txt` n'a pas changé — rebuild quasi instantané.

Sécurité : non-root

Ne jamais lancer l'API en `root`. Un `useradd` + `USER` réduit drastiquement la surface d'attaque.

Monitoring & Dérives (*Drift*)

Pourquoi mon modèle « meurt » en prod ?

- **Data Drift :** les données changent (ex: nouvel argot sur les réseaux sociaux)
- **Concept Drift :** le monde change (ex: prédictions avant/après une crise)
- **Infrastructure :** mise à jour de dépendance, changement de format d'entrée

Pipeline de surveillance

1. **Logger** les prédictions en production
2. **Collecter** la vérité terrain
3. **Comparer** distributions et métriques
4. **Alerter** si dégradation $>$ seuil
5. **Ré-entraîner** sur les nouvelles données

Outils

Evidently — rapports de drift données/modèle en HTML ou JSON

WhyLogs — profiling léger, intégrable directement dans la route API

MLflow — versioning des runs, comparaison de métriques entre modèles

Grafana — dashboards temps réel sur la latence et les erreurs applicatives

Stratégies de déploiement ML

Stratégies de déploiement

- **Canary :** 5 % du trafic sur le nouveau modèle — rollback immédiat si dégradation
- **A/B Testing :** deux modèles en parallèle, comparaison contrôlée sur population réelle
- **Shadow mode :** le nouveau modèle tourne en parallèle, logs seulement, zéro impact client

Outils courants

GitHub Actions ▪ **GitLab CI** ▪ **DVC** (versionner les données) ▪ **MLflow** (tracker les runs)

Règle d'or

Un modèle ne va jamais en prod sans avoir **passé les tests** et **été validé en staging**.

CI/CD ML — le pipeline en pratique

```yaml
# .github/workflows/ml-pipeline.yml
name: ML Pipeline
on:
  push:
    branches: [main]          # tests a chaque push
  schedule:
    - cron: "0 2 * * 1"      # retrain chaque lundi
  workflow_dispatch:          # retrain manuel

jobs:
  pipeline:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Tests preprocessing
        run: pytest tests/test_preprocessing.py
      - name: Eval modele
        run: python eval.py --min-f1 0.85
      - name: Build & push Docker
        run: docker build -t model:${{ github.sha }}
             . && docker push registry/model:${{
             github.sha }}
      - name: Deploy staging
        run: ./deploy.sh staging ${{ github.sha }}
```

Ce qui diffère du CI/CD web

- **Tests modèle :** accuracy, F1, latence — pas juste des assertions unitaires
- **Déclencheurs multiples :** push (tests), schedule (retrain), drift détecté (retrain d'urgence)
- **Artefacts lourds :** modèles de plusieurs gigaoctets — stocker dans un registry (MLflow, S3), pas dans git

Retrain $\neq$ redéploiement

Un retrain produit un **candidat** — à valider (tests + staging) avant promotion en production.

Récapitulatif — Ce qu'on a vu

Sérialisation & Optimisation

- Pickle : natif mais exécute du code arbitraire au chargement
- ONNX : graphe figé, portable (C++, JS, mobile)
- `state_dict` / `safetensors` : poids seuls, sûrs
- Quantization (Float32 → Int8), pruning, compilation hardware

Serving avec FastAPI

- Pattern Singleton via `lifespan` : modèle chargé une seule fois
- Pydantic : validation automatique des entrées
- Pipeline : JSON → preprocessing → inférence → JSON

Docker

- Images slim, `python:3.10-slim` comme base
- Layer caching : dépendances avant le code source
- Non-root : `useradd` + `USER` obligatoire en prod

Monitoring & CI/CD

- Data Drift / Concept Drift : surveiller les distributions en prod
- Pipeline CI/CD : tests unitaires → build Docker → staging → prod
- Déploiement : canary, A/B testing, shadow mode